Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

Cloud Computing (CS3081) Mini Project Report
Sandeep Kumar Saini, Roll No. 230101029
Indian Institute of Information Technology, Manipur
GitHub: https://github.com/sanju20434/cloud-computing-assignment

1. Introduction

A voice assistant has to turn a short sentence such as *set an alarm for seven thirty am tomorrow* into something a program can use. In this project the sentence is converted into a JSON object with four keys: `intent`, `date`, `time` and `place`. There are eight possible intents: alarm_set, alarm_query, calendar_set, calendar_query, calendar_remove, weather_query, datetime_query and recommendation_events. If a command does not mention a date, time or place, that slot is `null`.

Large language models can do this, but they are too heavy to run on my laptop. So I used **knowledge distillation**. A bigger teacher model (phi3.5, 3.8 billion parameters, run with Ollama) labels a set of commands, and a very small student model (EleutherAI/pythia-70m, 70 million parameters) is trained on those labels. The trained student is then packed into a Docker container and deployed on a local Kubernetes cluster created with k3d. The quality of the student is not the main goal of the project; the main goal is to practise the cloud side of the work.

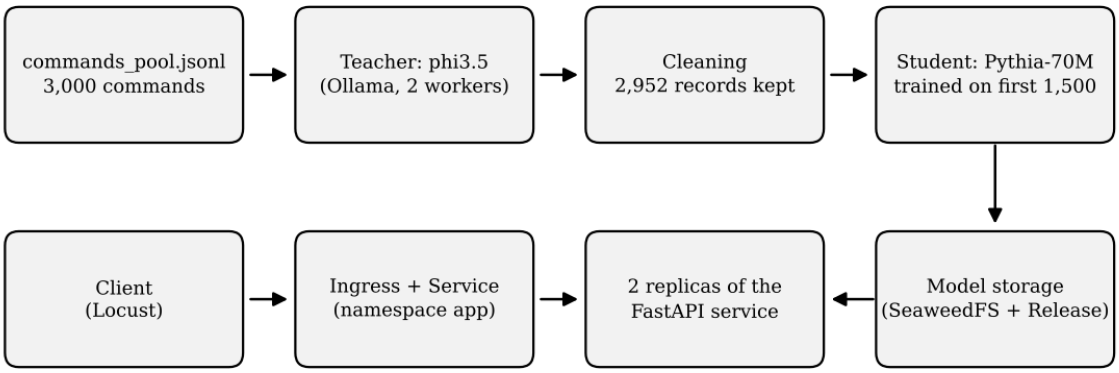


Figure 1. Overall pipeline of the project.

The report has one section for each of the seven tasks. The numbers come from the files in the repository. Where a part is still unfinished, I say so in the limitations of that section.

2. Task 1: Object storage and teacher labelling

2.1 What was done

I created a k3d cluster from `cluster/k3d.yaml` and made three namespaces: data (for storage), pipeline (for the labelling workers) and app (for the final service). The teacher is phi3.5 run through Ollama with temperature 0 and seed 42, using the prompt saved in `prompts/teacher_prompt.txt`. I did not edit the teacher's replies by hand.

2.2 Sharding

Labelling 3,000 commands on a CPU is slow, so the work is split between two workers. Each worker reads two environment variables, SHARD_INDEX and SHARD_COUNT, and takes the commands whose line number (starting from 0) leaves the remainder SHARD_INDEX when divided by SHARD_COUNT. With two workers, worker 0 gets the even lines and worker 1 gets the odd lines. Every command therefore belongs to exactly one worker, the workers do not need to talk to each other, and a restarted worker produces the same shard again. The raw replies are in `worker_0.jsonl` and `worker_1.jsonl`. Both workers use the same Ollama instance on one CPU, so the speed-up is smaller than 2 times.

Some commands failed in the first run. These 274 commands were run again and added to the dataset (the file before merging has 2,678 lines, after merging it has 2,952 lines).

2.3 Cleaning

The cleaning script checks every teacher reply. A record is removed if the reply is not valid JSON, if it does not have exactly the four keys, if the intent is not one of the eight, if a slot value is not null and does not appear in the command, or if the command is a duplicate. For every kept record a SHA-256 checksum of its JSON is stored.

Table 1. Dataset counts

Item	Count
Input records	3,000
Records removed (all reasons together)	48
Records kept in train.jsonl	2,952

The number removed for each separate reason is in the cleaning report; here only the total is given. The totals add up: 3,000 = 48 + 2,952.

2.4 Problems and limitations

- The file rejected.jsonl has 322 lines, but the cleaning report says 48 records were rejected. The rejected file is older than the other two files, so it probably comes from an earlier run. I used 48 because it matches 3,000 minus 2,952, but the rejected file should be regenerated.
- The teacher is not perfect. For example, the command "remind me of the meeting on tuesday" (C0003) got the intent alarm_set, and for C0005 the place is "with steve", although the rules say words like with, at and on should not be copied. I kept these records unchanged, as the assignment requires.
- The records in train.jsonl store the parsed JSON (fields id, command, output, sha256) and not the raw reply string that the assignment describes.

3. Task 2: Training the student

3.1 Method

This is sequence-level distillation. For each of the first 1,500 records in train.jsonl, the student sees the text "Command: <command>" followed by "JSON: <teacher reply>". The model is trained to predict the JSON part, so it learns to copy the teacher's answer. Training runs on the CPU with PyTorch and Hugging Face Transformers. All settings are stored in `training/config.yaml`: seed 42, batch size 2 with gradient accumulation 4, maximum length 128 tokens, and learning rate 0.0001 for 3 epochs (version V1). A second version V2 was trained for 5 epochs with learning rate 0.00005. The recorded training loss was 0.2508 for V1 and 0.1324 for V2.

3.2 A problem I had to fix

My first training runs gave very strange results: the loss was in the thousands, the gradient norm was infinite, a run of only 50 records took about 70 minutes, and the model only produced the text "UKUKUK". After checking, I found that the Pythia-70M weights load in float16. Loading the model in float32 solved it: the same 50-record run then took 29 seconds and the loss went down from 2.69 to 1.32.

3.3 Saved model files

Both models were uploaded to GitHub Release v1.0 as model-v1.zip and model-v2.zip, together with model_metadata.json. The weights are not committed to Git. The SHA-256 checksums of the two model.safetensors files, checked against my local files, are:

```
V1: F7C8D3412B23AD5006BA56D87FDEB324B3D4618B92203E5108FAA1959937B378
V2: F2BB8488963A883EBB3E84FD9DE7F10CCC14ECE95A9C8B3E2D6C0A7F92622E15
```

3.4 Results

The models were scored with tools/score.py on the 100 validation commands. These commands were never used for training.

Table 2. Validation scores

Metric (%)	Teacher	Untrained Pythia	Student V1	Student V2
JSON validity	100	-	100	100
Intent	73	-	70	74
Date	80	-	73	75
Time	83	-	78	76
Place	76	-	79	83
Full frame	40	-	39	40

The score of the untrained model is in results/baseline_validation.json and is added to this table by the build script.

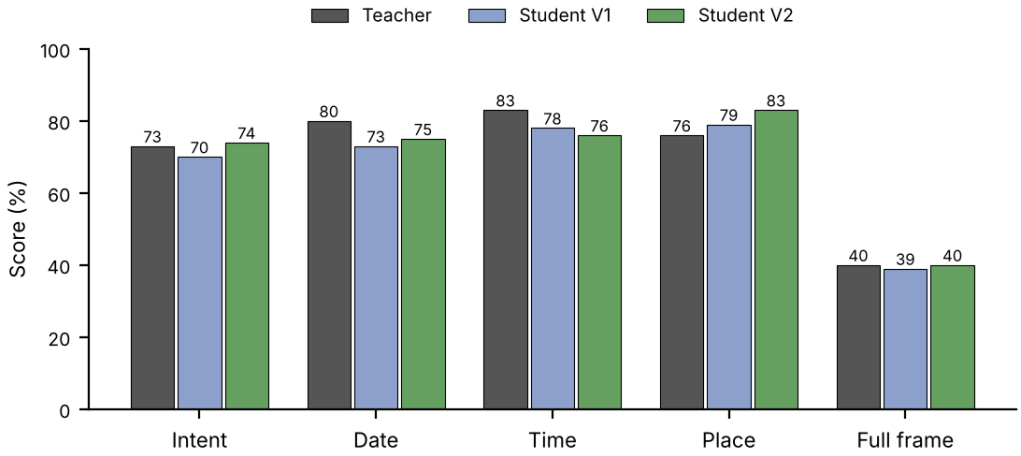


Figure 2. Scores of the teacher and the student models.

All models give valid JSON every time. V2 is the best student. It is better than the teacher in intent (74 against 73) and place (83 against 76), worse in date (75 against 80) and time (76 against 83), and equal in full-frame accuracy (40). V1 is a little lower than V2 on most metrics. Since there are only 100 examples, one point is one command, so differences of 1 to 3 points should not be taken too seriously. The student cannot be expected to be much better than its teacher, because it only learns from the teacher's answers and the teacher itself has only 40% full-frame accuracy.

3.5 Sequence-level and logit-based distillation

In **sequence-level** distillation the student is trained on the text that the teacher generated, as in this project. In **logit-based** distillation the student is trained to match the teacher's probability for every possible next token (the logits), which gives more information per example. This was not possible here for two reasons: Ollama only gave me the generated text, not the logits, and phi3.5 and Pythia use different tokenizers, so their token probabilities cannot be matched one to one. One limitation of my work is that the student copies the teacher's mistakes (see Section 2.4). Another is that the validation set is small, only 100 commands.

4. Task 3: The inference service

The trained model is served by a small FastAPI application that listens on port 8000. It has three endpoints:

- `POST /parse` takes a body like `{"text": "..."}` and returns the four-key JSON object. The response also has the headers X-Model-Version and X-Served-By (the pod name).
- `GET /healthz` returns 200 as long as the process is running.
- `GET /readyz` returns 503 until the model is loaded and 200 after that.

The service is packed in a Docker image without the model weights. During the load test the Locust output showed which pod answered each request and which model version it used (V2), so both headers work. The test command `make test` passes all the tests: 6 teacher tests, 8 intent tests and 6 NACL simulator checks.

Limitations. This report does not include the test results for the FAULT_INJECT switch, the exact error body for invalid output, the image size, and the 50-request run with 1 CPU and 1 GiB of memory with its peak memory. These should be shown with command output before

the final submission.

5. Task 4: Kubernetes deployment

The service runs as a Kubernetes Deployment with two replicas in the app namespace. A Service sends traffic to the pods and an Ingress sends the path /parse to the Service. The cluster is reached on http://localhost:8080. The endpoints /healthz and /readyz are meant for the liveness and readiness probes, so that a pod only receives traffic after the model is loaded.

Load balancing. With two ready replicas I ran the standard Locust test. Of the 245 successful requests, pod 1 served 149 (60.8%) and pod 2 served 96 (39.2%). Both are above the 30% required, and both pods reported model version V2.

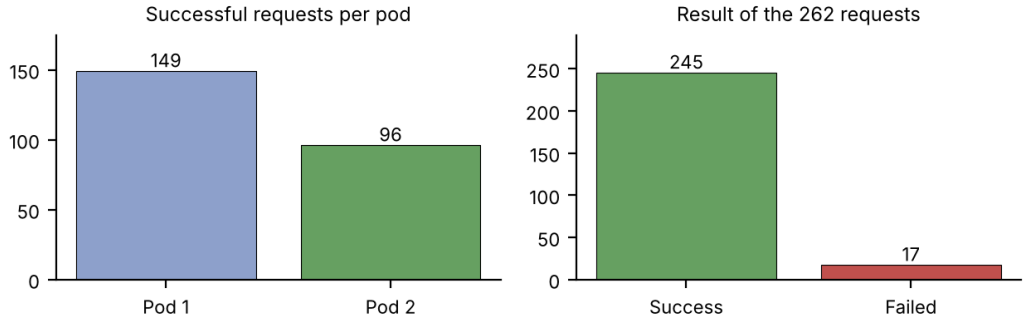


Figure 3. Distribution of requests between the two pods, and success and failure counts.

Cold start. I deleted all the pods and measured the time from pod creation until the pod was Ready, three times: 22.50 s, 22.43 s and 21.78 s. The median is **22.43 seconds**.

Rolling update, rollback and recovery. I changed MODEL_VERSION and MODEL_URI from v1 to v2 during a load test, rolled back with kubectl rollout undo, and deleted a pod while the load test was running. Kubernetes created a new pod each time. The numbers for the failed requests during these tests are not part of this report, and neither is the HPA scaling evidence (minimum 1, maximum 2 replicas, 70% CPU).

Important limitation. The whole cluster runs on one laptop. The k3d nodes are only containers inside one Docker Desktop VM (16 CPUs and 6.7 GiB of memory). So this setup shows that Kubernetes can replace a failed pod, but it does not show fault tolerance across machines: if the laptop stops, every replica stops. Also, the autoscaler uses the same shared CPU, so a second replica gives less extra capacity than it would on separate machines.

6. Task 5: Subnetting and network access control

The address block is 10.20.0.0/16. The subnets are made from the largest to the smallest, one after the other, with the smallest prefix that fits. The number of usable hosts is $2^{(32 - \text{prefix})} - 2$. For 1,000 hosts, $2^{10} = 1{,}024$ addresses are needed, which is a /22. For 250 hosts a /24 is needed (256), for 100 hosts a /25 (128) and for 20 hosts a /27 (32).

Table 3. Subnets

Subnet	Network	Usable hosts range	Broadcast	Usable hosts
app (1,000)	10.20.0.0/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022
public (250)	10.20.4.0/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data (100)	10.20.5.0/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management (20)	10.20.5.128/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

The NACL for the public subnet (where the load balancer is) is below. Rules with a lower number are checked first and the first match decides. If nothing matches, the packet is denied.

Table 4. NACL rules for the public subnet

Direction	Rule	Protocol and port	Address	Action	Reason
Inbound	100	all	198.51.100.0/24	deny	blocked abusive range
Inbound	110	TCP 80	0.0.0.0/0	allow	web traffic
Inbound	120	TCP 443	0.0.0.0/0	allow	secure web traffic
Inbound	130	TCP 22	10.20.5.128/27	allow	SSH from management
Inbound	140	TCP 1024-65535	10.20.0.0/22	allow	replies from app subnet
Outbound	100	TCP 8000	10.20.0.0/22	allow	requests to app subnet
Outbound	110	TCP 1024-65535	0.0.0.0/0	allow	replies to clients

I checked the rules with tools/nacl_sim.py on the six sample packets and all six were decided correctly.

Why replies need their own rules. A NACL is stateless. It does not remember connections, so it looks at every packet alone. Even if a request was allowed in, the reply going out is a new packet and needs an allow rule of its own. The clients use ephemeral ports from 1024 to 65535, so those ports are opened for the replies.

Why the order matters. The first matching rule wins. The deny rule for 198.51.100.0/24 has number 100, which is lower than the allow rules for ports 80 and 443. If it had a higher number, a packet from 198.51.100.9 to port 443 would match the allow rule first and the block would never work.

7. Task 6: Reproducibility, testing and CI/CD

All code, tests, configuration files and Kubernetes manifests are kept in the GitHub repository. The model weights are not in Git; they are in the GitHub Release. The file AI_USE.md describes which AI tools I used and how I checked their output.

The GitHub Actions workflow (.github/workflows/ci.yml) currently does the following: it checks out the code, sets up Python 3.12, installs the packages, compiles the Python files, runs the teacher tests (6 passed), runs the intent tests (8 passed) and runs the NACL simulator (6 of 6 correct).

What is still missing. The workflow does not yet have the linting stage, the model evaluation that downloads the model from the GitHub Release, the quality gate with `--min-validity 95`, and the gitleaks secret scan. I also still need the link to a deliberately failed run (using the

untrained Pythia model) to show the gate working. Another problem is that the workflow is set to run on the branch main, but my pushed branch is called master, so it may not start at all. The next steps are to add the missing stages in the order lint, unit tests, evaluation, quality gate, fix the branch name, and put the failed run link into the final report.

8. Task 7: Performance, cost and review

8.1 Load test

I ran the standard Locust test: 4 users, spawn rate 4 per second, 120 seconds, against http://localhost:8080, using the commands from validation.jsonl.

Table 5. Load test results

Measurement	Value
Total requests	262 (245 successful, 17 failed)
Error rate	6.49% (all 17 failures were HTTP 500)
Median (p50) latency	1,700 ms
p95 latency	3,300 ms
Average latency	1,813 ms
Throughput	about 2.2 requests per second
kubectl top, pod 1	3m CPU, 403 MiB memory
kubectl top, pod 2	2m CPU, 724 MiB memory
Cold start (Task 4)	median 22.43 s

The cause of the 17 HTTP 500 errors is not found yet. It could be that the service rejected invalid model output, or that the pods were short of time or memory under load (pod 2 used 724 MiB, which is about 71% of a 1 GiB limit). The CPU numbers (2m and 3m) are very low for a busy pod, so the kubectl top snapshot was probably not taken at the busiest moment.

8.2 Cost

The rate card gives 3.50 INR per vCPU-hour and 0.45 INR per GiB-hour. I assume each replica has limits of 1 vCPU and 1 GiB, which gives the 7.90 INR per hour that I recorded.

- One replica per hour: 1 x 3.50 + 1 x 0.45 = 3.95 INR.
- Two replicas per hour: 2 x 3.95 = 7.90 INR.
- Requests per hour: 2.2 x 3,600 = 7,920.
- Cost per 1,000 requests: 7.90 / 7,920 x 1,000 = 0.997, so about **1.00 INR**. If only the 245 successful requests are counted (7,350 per hour), the cost is 1.07 INR.

8.3 SLO and error budget

My SLO is: *95% of the /parse requests must return HTTP 200 within 30 seconds*, measured over one load-test run (262 requests). The error budget is 5% of 262 = 13.1, so 13 failed requests are allowed. There were 17 failures, so the whole budget was used up (about 131%) and it was exceeded by 4 requests. The SLO was **not met**: the success rate was 93.51%. The p95 latency (3,300 ms) is below the 5,000 ms limit of the assignment, but the error rate (6.49%) is above its 1% limit.

8.4 Review

Table 6. Review of the system

Part	Measured number	Limitation	Improvement
Data and storage	2,952 of 3,000 records kept	Rejected file does not match the report	Regenerate the counts in one clean run
Model quality	Full frame 40% (teacher also 40%)	Teacher mistakes are copied; only 100 validation examples	Better labels, more validation data
Latency and throughput	p95 3,300 ms, 2.2 requests/s	Slow CPU inference	Fewer output tokens, stop at the closing brace, quantisation
Resource use	403 MiB and 724 MiB memory	Only one snapshot, not the peak	Record kubectl top during the whole test
Reliability	17 of 262 requests failed	One laptop only, errors not explained	Log failed outputs, use more than one node
Security and cost	about 1.00 INR per 1,000 requests	No secret scan in CI yet	Add gitleaks, keep keys in a Secret

9. Conclusion

In this project I generated labels with a teacher model, cleaned them, trained a 70-million-parameter student on them, put it in a container and ran it on a local Kubernetes cluster. The best student (V2) reached 100% valid JSON and 40% full-frame accuracy, the same as the teacher. The deployment worked with two replicas and a cold start of about 22 seconds, but the load test had 6.49% failed requests, which is above the 95% success target. The things I still want to finish are the explanation of the HTTP 500 errors, the missing CI stages, and the evidence for the Task 3 tests. This project taught me how the different parts of a cloud system, from data to deployment, depend on each other.

References

[1] Cloud Computing (CS3081), Mini Project assignment brief.

[2] Y. Kim and A. M. Rush, Sequence-Level Knowledge Distillation, EMNLP 2016. https://aclanthology.org/D16-1139/

[3] E. Bastianelli et al., SLURP, EMNLP 2020. https://aclanthology.org/2020.emnlp-main.588/

[4] EleutherAI, Pythia-70M model card. https://huggingface.co/EleutherAI/pythia-70m

[5] Ollama, phi3.5. https://ollama.com/library/phi3.5 [6] k3d. https://k3d.io/stable/ [7] Locust. https://locust.io

[8] AWS VPC User Guide, Network ACLs. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html

[9] Google SRE Book, Service Level Objectives. https://sre.google/sre-book/service-level-objectives/